8253jang@daum.net
장희정

React Redux는

React 애플리케이션에서 상태 관리를 효율적으로 하기 위한 라이브러리로 Redux의 전역 상태 저장소를 React 컴포넌트에서 쉽게 사용할 수 있게 도와준다. 상태를 중앙에서 관리함으로써 예측 가능한 상태 흐름과 디버깅이 가능해진다.

용어 정리

1. Redux(https://redux.js.org/)

Redux는 **자바스크립트** 앱에서 상태를 예측 가능하게 관리하기 위한 **상태 관리 라이브러리**
React뿐만 아니라 Vanilla JS, Angular, Vue 등 어떤 UI 라이브러리/프레임워크와도 사용 가능

핵심 개념

- **Store :** 상태를 저장하는 중앙저장소
- **Action :** 상태 변경 요청 객체 (type , payload)
- **Reducer :** 상태를 변경하는 순수함수
- **Dispatch :** action을 스토어에 전달하는 함수

2. React-Redux(https://react-redux.js.org/)

Redux를 React에서 쉽게 연동할 수 있도록 도와주는 공식 바인딩 라이브러리
React 컴포넌트와 Redux store를 연결해 주는 역할

주요 기능

- <Provider store={store}>: 전역 상태를 리액트 컴포넌트에 주입
- **useSelector():** store의 상태를 읽을 수 있게 함
- **useDispatch():** action을 dispatch할 수 있게 함

3. Redux Toolkit (RTK - https://redux-toolkit.js.org/)

Redux의 복잡한 설정과 보일러플레이트 코드를 줄이기 위해 Redux 팀이 공식적으로 만든 권장 도구 모음

- Redux의 설정을 단순화
- 자동으로 action 생성자, reducer 생성, 불변성 유지 등 제공
- **configureStore() , createSlice(), createAsyncThunk()** 등의 API 제공

8253jang@daum.net
장희정

정리를 하면,

세 가지는 각각의 역할이 다르지만, 함께 사용되며 상호보완적으로 협력하는 관계이다.

Redux = 요리법의 원칙과 구조
React-Redux = 주방과 식탁을 연결하는 서빙 도구 (식탁 = React 컴포넌트)
Redux Toolkit = 주방 도구 세트 (요리를 쉽게 만드는 도구들)

라이브러리	주요 역할	다른 라이브러리와의 관계
Redux	상태 관리의 원칙과 구조 제공 (Store, Action, Reducer 등)	**기반 철학과 로직** 제공, React와 직접 연결하지 않음
React-Redux	Redux를 React에 연결 (Provider, useSelector, useDispatch)	**Redux와 React를 연결**해주는 다리 역할
Redux Toolkit	Redux의 보일러플레이트를 줄이고 쉽게 쓰게 도와주는 도구 모음	**Redux를 간편하게 구현**할 수 있게 도와줌

React Context vs Redux 차이점

항목	React Context	Redux
목적	간단한 상태 공유	복잡한 상태 로직 및 전역 상태 관리
상태 저장 위치	컴포넌트 트리 내부	독립된 **중앙 저장소** (store)
상태 업데이트	setState, useReducer 사용	액션(Action) → 리듀서(Reducer) → 상태 변경
성능	잦은 렌더링 발생 (전체 Provider 영향)	**성능 최적화에 유리** (selective update)
구조	간단한 구조	명확한 구조와 미들웨어 활용 가능

Context는 간단한 전역 상태에 적합,
Redux는 복잡한 상태나 여러 컴포넌트가 공유하는 데이터에 적합

패키지 설치

```
npm install @reduxjs/toolkit react-redux
```

8253jang@daum.net
장희정

실습 1

```
# 프로젝트 생성
  npm create vite@latest step20-redux

# 폴더 이동
  cd step20-redux

# package 설치
 npm i
 npm install @reduxjs/toolkit react-redux

# App.css / index.css 내용 비우기
```

코드 작성하가

App.jsx

```
import './App.css'
function App() {
  return (
    <>
      <h1>React-Redux TEST</h1>

      {/* 카운트 UI 영역 */}
      <div className="Count">
        {/* 버튼 클릭 시 각각 액션을 디스패치 */}
        <button>빼기</button>

        {/* 현재 Redux store에 저장된 상태 값 표시 */}
        <span> 숫자 </span>

        <button>더하기</button>
        {/* 5만큼 증가시키는 커스텀 액션 실행 */}
        <button>5씩 증가</button>
      </div>
    </>
  )
}

export default App
```

8253jang@daum.net
장희정

App.css	.Count{
	display: flex;
	gap: 10px;
	}

실행화면	#실행하기 npm run dev

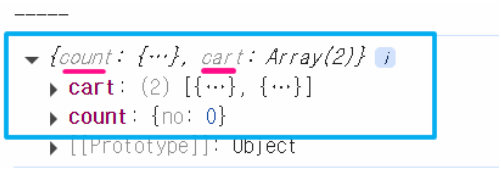
Redux 사용하기

1. 저장소 만들기(src/store.js)
2. Provider 적용하기 (src/main.jsx)
3. App.jsx에서 store.js 변수 꺼내 쓰기

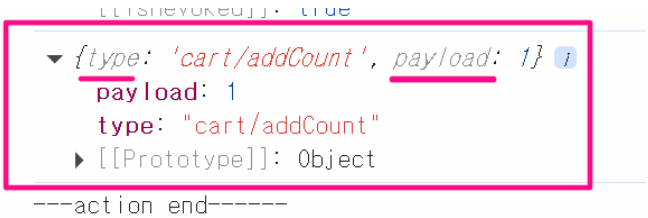
Redux사용 문법	**import { configureStore, createSlice } from '@reduxjs/toolkit'**
	
	☞ **문법**(Slice – 작은 store의 개념)
	const slice변수이름 = createSlice({
	name : , //slice이름(이 이름은 액션 타입에 prefix로 사용됨)
	initialState : , //초기값
	reducers: { // 객체안에 상태를 변경하는 로직과 해당 액션정의(함수정의)
	함수이름(state, action){ } , //action에는 type, payload (ex: action.payload)
	함수이름( ){ } ,
	
	}
	});
	
	☞ **문법(위의 reducers에서 정의 된 액션(함수) 외부에서 사용 할 수 있도록 export**
	export const { 함수이름, 함수이름, ..} = slice변수이름.actions;
	
	☞ **configureStore를 사용하여 Redux 스토어 생성(여러 slice를 reducer에 등록할수 있다)**
	export default configureStore({
	reducer: {
	이름 : slice변수이름.reducer, //
	}

<table>
<tr>
<td></td>
<td>
});

☞ 문법 (외부component에서 사용하기)
const state = useSelector((state) => state)
console.log(state);



☞ 문법 (외부component에서 함수 호출하기)
let dispatch = useDispatch()
dispatch(함수이름(payload정보));



</td>
</tr>
<tr>
<td>src> store.js
코딩하기</td>
<td>
// Redux Toolkit에서 store를 설정하는 configureStore와 slice를 생성하는 createSlice를 import
import { configureStore, createSlice } from '@reduxjs/toolkit'

/*
createSlice() : reducer + action을 한번에 생성
configureStore() : Redux store를 생성하며 여러 slice를 통합
state.count.no : 컴포넌트에서 이 값으로 상태 접근 가능
dispatch(up()) : 액션을 실행하는 방법
*/

// createSlice 함수는 Redux의 action + reducer를 하나의 객체로 간단하게 정의할 수 있게 해줌
const countSlice = createSlice({
    // slice의 이름. 이 이름은 액션 타입에 prefix로 사용됨 (예: count/up)
    name: "count",

    // 초기 상태 정의 (기본 상태 값)
    initialState: { no: 0 },

    // reducers 객체 안에 상태를 변경하는 로직과 해당 액션을 정의
    reducers: {
        // 'up' 액션: 상태의 no 값을 1 증가
        up(state) {
</td>
</tr>
</table>

```
            state.no = state.no + 1;
        },

        // 'down' 액션: 상태의 no 값을 1 감소
        down(state) {
            state.no = state.no - 1;
        },

        // 'incrementByAmount' 액션: 전달된 payload만큼 상태의 no 값을 증가
        incrementByAmount(state, action) {
            // action.payload는 dispatch 시 전달된 값
            state.no = state.no + action.payload;
        }
    }
});

// 위 reducers에 정의된 액션 생성자(action creator)를 추출하여 export
// 나중에 dispatch(up()), dispatch(down()) 등으로 사용할 수 있음
export const { up, down, incrementByAmount } = countSlice.actions;

// configureStore를 사용하여 Redux 스토어 생성
// 여러 slice를 reducer에 등록할 수 있으며, 여기서는 count 하나만 사용
export default configureStore({
    reducer: {
        count: countSlice.reducer, // state.count로 접근 가능

    }
});
```

8253jang@daum.net
장희정

work > step21-redux > src > JS store.js > ...

```javascript
// Redux Toolkit에서 store를 설정하는 configureStore와 slice를 생성하는 createSlice를 import
import { configureStore, createSlice } from '@reduxjs/toolkit'

/*
createSlice() : reducer + action을 한번에 생성
configureStore() : Redux store를 생성하며 여러 slice를 통합
state.count.no : 컴포넌트에서 이 값으로 상태 접근 가능
dispatch(up()) : 액션을 실행하는 방법
*/

// createSlice 함수는 Redux의 action + reducer를 하나의 객체로 간단하게 정의할 수 있게 해줌
const countSlice = createSlice({
    // slice의 이름. 이 이름은 액션 타입에 prefix로 사용됨 (예: count/up)
    name: "count",

    // 초기 상태 정의 (기본 상태 값)
    initialState: { no: 0 },

    // reducers 객체 안에 상태를 변경하는 로직과 해당 액션을 정의
    reducers: {
        // 'up' 액션: 상태의 no 값을 1 증가
        up(state) {
            state.no = state.no + 1;
        },

        // 'down' 액션: 상태의 no 값을 1 감소
        down(state) {
            state.no = state.no - 1;
        },

        // 'incrementByAmount' 액션: 전달된 payload만큼 상태의 no 값을 증가
        incrementByAmount(state, action) {
            // action.payload는 dispatch 시 전달된 값
            state.no = state.no + action.payload;
        }
    }
});

// 위 reducers에 정의된 액션 생성자(action creator)를 추출하여 export
// 나중에 dispatch(up()), dispatch(down()) 등으로 사용할 수 있음
export const { up, down, incrementByAmount } = countSlice.actions;

// configureStore를 사용하여 Redux 스토어 생성
// 여러 slice를 reducer에 등록할 수 있으며, 여기서는 count 하나만 사용
export default configureStore({
    reducer: {
        count: countSlice.reducer, // state.count로 접근 가능

    }
});
```

8253jang@daum.net
장희정

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import { Provider } from 'react-redux';
import store from './store.js';
createRoot(document.getElementById('root')).render(

  <StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </StrictMode>,
)
```

```
1   import { StrictMode } from 'react'
2   import { createRoot } from 'react-dom/client'
3   import './index.css'
4   import App from './App.jsx'
5   import { Provider } from 'react-redux';
6   import store from './store.js';
7   createRoot(document.getElementById('root')).render(
8
9     <StrictMode>
10      <Provider store={store}>
11        <App />
12      </Provider>
13    </StrictMode>,
14  )
15
```

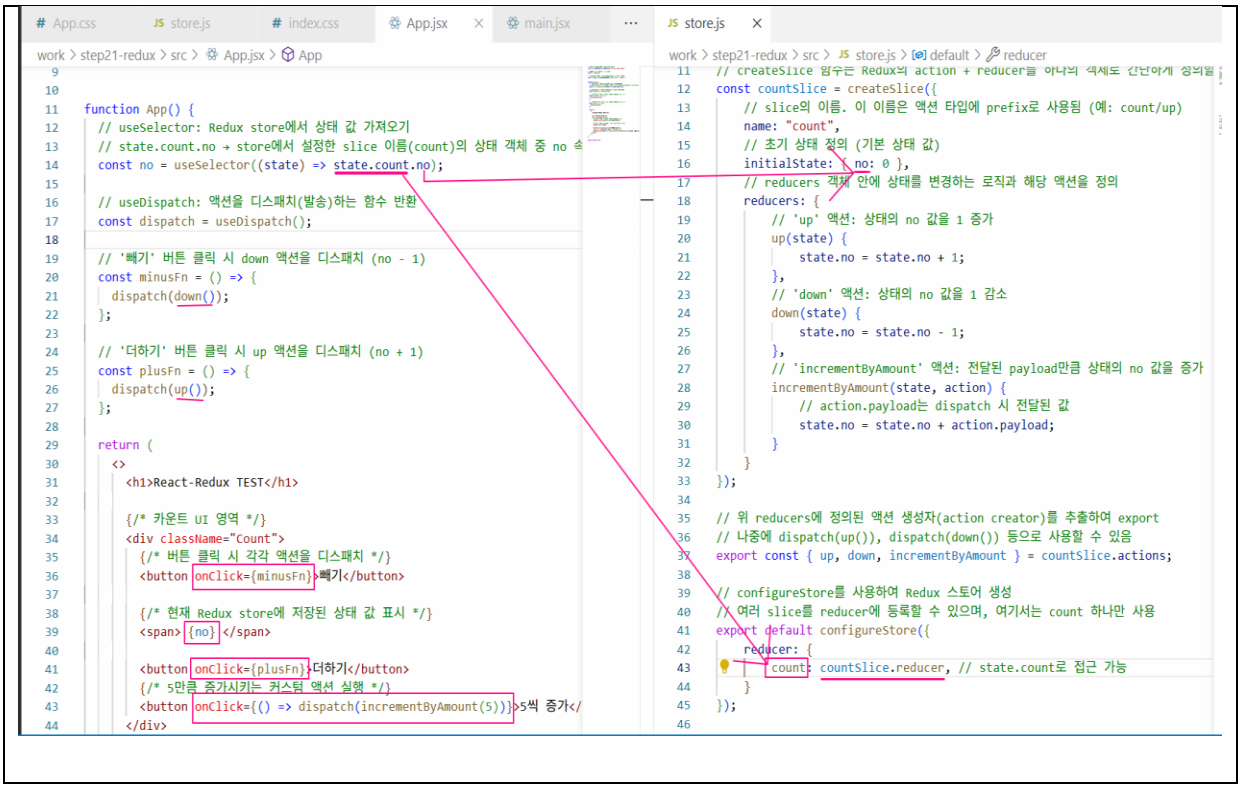
<Provider store={import해온거}>
이걸로 <App/> 을 감싸면
이제 <App>과 그 모든 자식컴포넌트들은 store.js에 있던 state를 맘대로 꺼내 쓸 수 있다.

Redux의 App.jsx에서 store.js 변수를 꺼내 쓰기

```
// Redux store와 연결된 React 훅 import
import { useDispatch, useSelector } from 'react-redux'

// 스타일 파일 import (선택 사항)
import './App.css'

// store에서 정의한 액션 생성자(import한 slice 액션들)
import { down, incrementByAmount, up } from './store';

function App() {
```

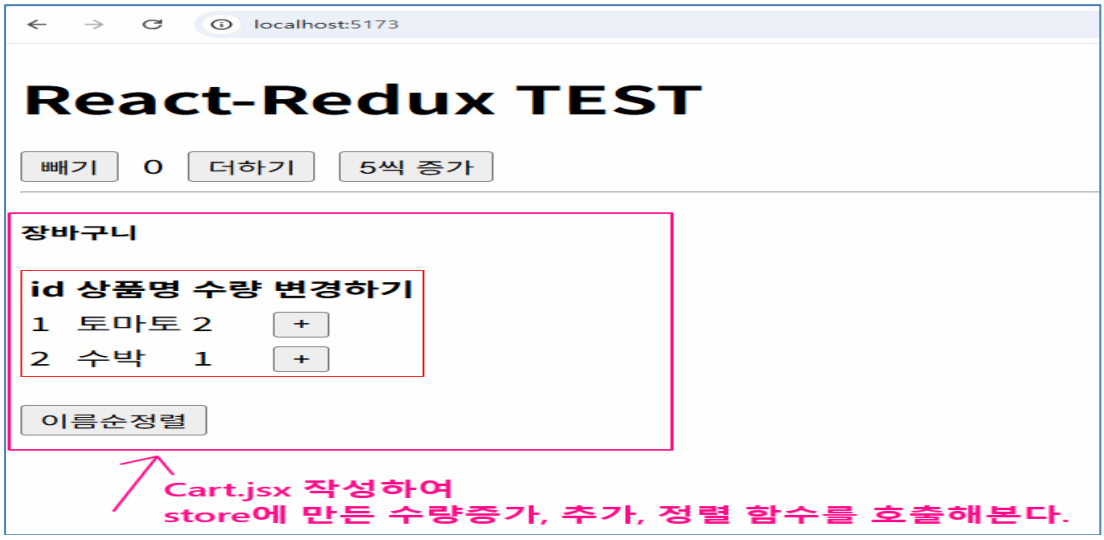
```
  // useSelector: Redux store에서 상태 값 가져오기
  // state.count.no → store에서 설정한 slice 이름(count)의 상태 객체 중 no 속성
  const no = useSelector((state) => state.count.no);

  // useDispatch: 액션을 디스패치(발송)하는 함수 반환
  const dispatch = useDispatch();

  // '빼기' 버튼 클릭 시 down 액션을 디스패치 (no - 1)
  const minusFn = () => {
    dispatch(down());
  };

  // '더하기' 버튼 클릭 시 up 액션을 디스패치 (no + 1)
  const plusFn = () => {
    dispatch(up());
  };

  return (
    <>
      <h1>React-Redux TEST</h1>

      {/* 카운트 UI 영역 */}
      <div className="Count">
        {/* 버튼 클릭 시 각각 액션을 디스패치 */}
        <button onClick={minusFn}>빼기</button>

        {/* 현재 Redux store에 저장된 상태 값 표시 */}
        <span> {no} </span>

        <button onClick={plusFn}>더하기</button>
        {/* 5만큼 증가시키는 커스텀 액션 실행 */}
        <button onClick={() => dispatch(incrementByAmount(5))}>5씩 증가</button>
      </div>
    </>
  )
}

export default App
```

8253jang@daum.net
장희정



실습 2

Cart.jsx 컴포넌트를 만들어 store에 작성한 수량증가, 정렬 함수를 호출해보자.



8253jang@daum.net
장희정

store.js수정

```
# store.js 파일의 export const { up, down, incrementByAmount } = countSlice.actions;  부분 뒤에
# 아래의 코드를 추가한다.


let cartSlice = createSlice({
    name : 'cart',
    initialState : [
      {id : 1, name : '토마토', count : 2},
      {id : 2, name : '수박', count : 1}
    ],
    reducers : {
        addCount(state, action){
            let num = state.findIndex((item)=>{ return item.id === action.payload })
            console.log(num);
            state[num].count++;
        },
        addItem(state, action){
             state.push(action.payload)
        },

        sortName(state) {
          state.sort((a, b) =>
            a.name > b.name ? 1 : -1,
          );
       }
    }
  })

export let { addCount, addItem,   sortName } = cartSlice.actions


# export default configureStore 부분에  cart slice 를 reducer에 등록 한다.

export default configureStore({
    reducer: {
        count: countSlice.reducer, // state.count로 접근 가능
        cart : cartSlice.reducer
    }
});
```

8253jang@daum.net
장희정

```
// configureStore를 사용하여 Redux 스토어 생성
// 여러 slice를 reducer에 등록할 수 있으며, 여기서는 count 하나만 사용
export default configureStore({
    reducer: {
        count: countSlice.reducer, // state.count로 접근 가능
        cart : cartSlice.reducer
    }
});
```

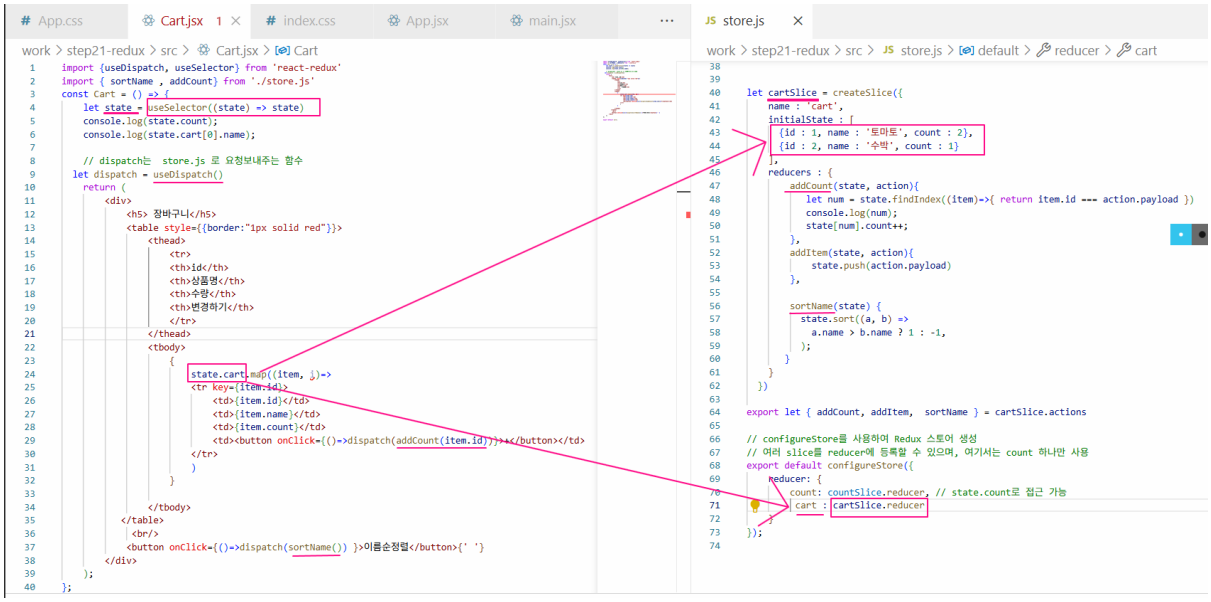
이 부분 추가 한다.

Cart.jsx

```
import {useDispatch, useSelector} from 'react-redux'
import { sortName , addCount} from './store.js'
const Cart = () => {
    let state = useSelector((state) => state)
    console.log(state.count);
    console.log(state.cart[0].name);

    // dispatch는  store.js 로 요청보내주는 함수
  let dispatch = useDispatch()
    return (
        <div>
            <h5> 장바구니</h5>
            <table style={{border:"1px solid red"}}>
                <thead>
                    <tr>
                    <th>id</th>
                    <th>상품명</th>
                    <th>수량</th>
                    <th>변경하기</th>
                    </tr>
                </thead>
                <tbody>
                    {
                        state.cart.map((item, i)=>
                        <tr key={item.id}>
                            <td>{item.id}</td>
                            <td>{item.name}</td>
                            <td>{item.count}</td>
                            <td><button onClick={()=>dispatch(addCount(item.id))}>+</button></td>
                        </tr>
                        )
                    }

                </tbody>
            </table>
              <br/>
```

8253jang@daum.net
장희정

```
                <button onClick={()=>dispatch(sortName()) }>이름순정렬</button>{' '}
            </div>
        );
};

export default Cart;
```



App.jsx 컴포넌트에 Cart.jsx 컴포넌트 넣기

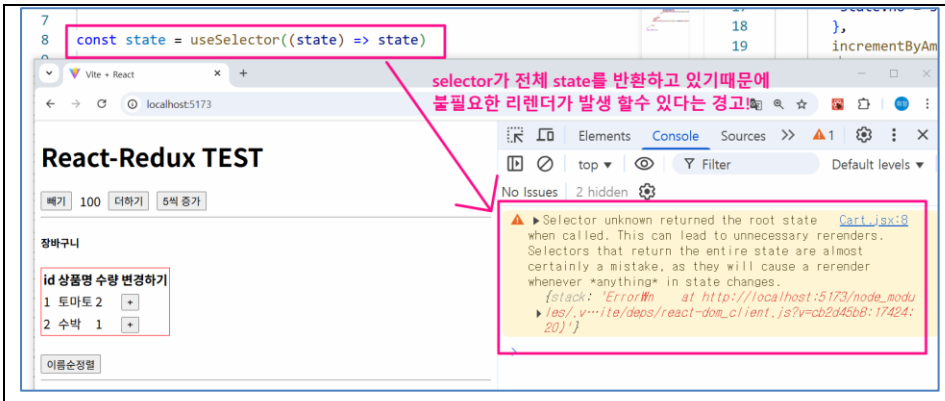
App.jsx 컴포넌트의 return 부분에 아래의 코드 추가

```
<hr/>
<Cart/>
```

```
43            <button onClick={() => dispatch(incrementByAmount(5))}>5씩 증가</button>
44          </div>
45
46          <hr/>
47          <Cart/>
48
49        </>
50      )
51    }
52
53    export default App
```

알아두기

8253jang@daum.net
장희정



☞ 문제의 원인

Redux에서 useSelector를 사용할 때, 전체 state를 그대로 반환하면, store의 어떤 값 하나라도 바뀔 때마다 컴포넌트가 리렌더링되기 때문에 성능에 문제가 생길 수 있다.

```
const state = useSelector((state) => state); // ❌ 전체 state 반환
```

이렇게 하면 store의 어떤 값이 바뀌든 무조건 컴포넌트가 다시 렌더링된다.
React-Redux는 이전 state와 새로운 state를 얕은 비교(shallow equal)로 비교하기 때문에 전체 객체가 바뀌었다고 인식한다.

☞ 해결 방법

useSelector에서 필요한 slice(부분 state)만 선택

```
// count라는 slice에서 필요한 부분만 선택
const count = useSelector((state) => state.count.value);
```

만약,

store에 있는 state를 2개 이상 동시에 가져오고 싶다면, useSelector를 여러 번 쓰거나,
한 번의 useSelector에서 필요한 값만 추출하는 방식 둘 다 가능하다.

방법 1 : useSelector를 여러 번 사용하는 방법

```
const count = useSelector((state) => state.counter.value);
const user = useSelector((state) => state.user.name);
```

- 이 방식은 각 selector가 독립적으로 작동하므로 불필요한 리렌더를 최소화할 수 있다.
- React-Redux는 각각의 selector 결과가 바뀔 때만 리렌더를 유발한다.

방법 2 : useSelector 한 번으로 여러 값 가져오기

8253jang@daum.net
장희정

```
const { value: count, name: userName } = useSelector((state) => ({
  value: state.counter.value,
  name: state.user.name
}));
```

이렇게 하면 한 번에 필요한 값들만 묶어서 가져올 수 있다. 단, 이 방식은 객체를 반환하므로 매번 새로운 객체가 생성되어 리렌더링이 발생할 수 있다.
이를 성능 최적화하려면 shallowEqual을 같이 써주는 것이 좋다.

```
import { shallowEqual, useSelector } from 'react-redux';

const { value: count, name: userName } = useSelector(
  (state) => ({
    value: state.counter.value,
    name: state.user.name
  }),
  shallowEqual
);
```

☞ **정리**

방법	장점	비고
`useSelector` 여러 번	간단하고 성능 좋음	추천 방법
한 번에 여러 값	코드 간결	`shallowEqual`을 꼭 써야 불필요한 리렌더 방지 가능

개선코드

```
const { stateCart, stateCount }= useSelector((state) => {
    return  {stateCart : state.cart , stateCount : state.count}
} ,  shallowEqual) ;
```

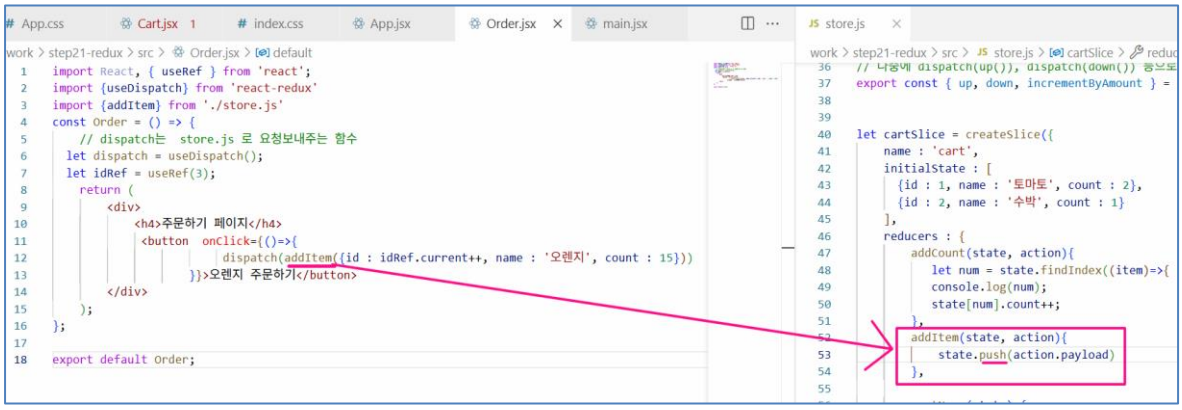
8253jang@daum.net
장희정

실습 3

Order.jsx 컴포넌트를 만들어 store에 작성한 Cart에 item 추가 하는 함수를 호출해보자.

Order.jsx 작성

```
import React, { useRef } from 'react';
import {useDispatch} from 'react-redux'
import {addItem} from './store.js'
const Order = () => {
    // dispatch는  store.js 로 요청보내주는 함수
  let dispatch = useDispatch();
  let idRef = useRef(3);
    return (
        <div>
            <h4>주문하기 페이지</h4>
             <button  onClick={()=>{
                        dispatch(addItem({id : idRef.current++, name : '오렌지', count : 15}))
                    }}>오렌지 주문하기</button>
        </div>
    );
};

export default Order;
```



App.jsx파일에 Order.jsx 컴포넌트 추가한다.
<Cart/> 컴포넌트 뒤에 추가한다.

```
<hr/>
<Order/>
```

8253jang@daum.net
장희정

```
46          <hr/>
47          <Cart/>
48
49            <hr/>
50          <Order/>
51
52        </>
53      )
54    }
55
56    export default App
```

React Redux가 **성능 최적화에 유리하다**는 말은, **컴포넌트가 필요로 하는 상태만 선택적으로 구독 (Selective subscription)** 하여, **불필요한 렌더링을 방지**할 수 있다는 뜻이다.

어떤 상황에서 유리한가?

Context API(전파형) vs Redux(구독형)

상황	Context API	Redux
전체 state가 Provider에 있음	Provider 값이 바뀌면, 하위 컴포넌트 **모두 리렌더링**	필요한 부분만 `useSelector` 로 구독해서 **선택적 리렌더링** 가능

예제: 컴포넌트가 일부 상태만 사용할 때

```
// store.js
import { configureStore, createSlice } from "@reduxjs/toolkit";

const userSlice = createSlice({
  name: "user",
  initialState: { name: "Grace", age: 30 },
  reducers: {
    setName: (state, action) => {
      state.name = action.payload;
    },
    setAge: (state, action) => {
      state.age = action.payload;
    },
  },
});

export const { setName, setAge } = userSlice.actions;
export const store = configureStore({
  reducer: { user: userSlice.reducer },
});
```

8253jang@daum.net
장희정

```jsx
// App.jsx
import { Provider } from "react-redux";
import { store } from "./store";
import NameDisplay from "./NameDisplay";
import AgeDisplay from "./AgeDisplay";

function App() {
  return (
    <Provider store={store}>
      <NameDisplay />
      <AgeDisplay />
    </Provider>
  );
}

export default App;
```

```jsx
// NameDisplay.jsx
import { useSelector, useDispatch } from "react-redux";
import { setName } from "./store";

const NameDisplay = () => {
  const name = useSelector((state) => state.user.name); // name만 구독
  const dispatch = useDispatch();

  return (
    <div>
      <p>이름: {name}</p>
      <button onClick={() => dispatch(setName("Eunji"))}>이름 변경</button>
    </div>
  );
};

export default NameDisplay;
```

8253jang@daum.net
장희정

```jsx
// AgeDisplay.jsx
import { useSelector, useDispatch } from "react-redux";
import { setAge } from "./store";

const AgeDisplay = () => {
  const age = useSelector((state) => state.user.age); // age만 구독
  const dispatch = useDispatch();

  return (
    <div>
      <p>나이: {age}</p>
      <button onClick={() => dispatch(setAge(age + 1))}>나이 증가</button>
    </div>
  );
};

export default AgeDisplay;
```

결과

setName을 눌러도 AgeDisplay는 **리렌더링되지 않음**
setAge를 눌러도 NameDisplay는 **리렌더링되지 않음**

왜 성능이 좋은가?

- useSelector는 **state의 특정 조각만 구독**하고, 해당 값이 바뀔 때만 리렌더링
- 상태가 커져도 **관심 있는 조각만 구독해서 렌더링 비용 감소**
- 필요 없는 컴포넌트까지 리렌더링되는 Context API보다 **더 세밀한 제어 가능**